

Confidence-Based Active Data Imputation via Distribution Matching [Regular]

Zarin Tahia Hossain

Department of Computer Science, Western University
London, Ontario, Canada
zarin.hossain@uwo.ca

Mostafa Milani

Department of Computer Science, Western University
London, Ontario, Canada
mostafa.milani@uwo.ca

ABSTRACT

Data imputation is essential for handling missing values in datasets, but fully automated methods often fail to incorporate human expertise, which can significantly improve imputation quality in practical settings. Active imputation addresses this limitation by leveraging user interactions to guide the process, particularly when automated methods produce uncertain results. In this paper, we propose a novel active imputation framework that combines distribution matching with confidence estimation to effectively guide user input.

We define a confidence measure for imputed values by computing the standard deviation from multiple runs of OT-Impute. To reduce the computational cost of multiple runs, we introduce an efficient approach using dropout noise injection, which approximates confidence through variability from a single stochastic run. This approach significantly lowers computational cost while maintaining reliable confidence estimation.

We present two active imputation methods: Batch-Impute, which selects low-confidence imputations in a single step, and Iter-Impute, an iterative method that allocates the budget over multiple iterations, progressively enhancing imputations with user feedback. Our experiments on real-world datasets demonstrate that Iter-Impute consistently outperforms both Batch-Impute and random selection, achieving higher accuracy while efficiently utilizing human input. This makes our approach well-suited for semi-automated data cleaning scenarios where human involvement is limited but crucial.

VLDB Workshop Reference Format:

Zarin Tahia Hossain and Mostafa Milani. Confidence-Based Active Data Imputation via Distribution Matching [Regular]. VLDB 2025 Workshop: Quality in Databases (QDB'25).

VLDB Workshop Artifact Availability:

The source code, data, and/or other artifacts have been made available at https://github.com/ZarinTahia/active_imputer.

1 INTRODUCTION

Imputation is a fundamental task in data preprocessing, aimed at handling missing values in datasets. High-quality imputation is crucial, as missing data can significantly degrade the performance of downstream applications. A wide range of imputation techniques

have been developed, ranging from simple heuristics to sophisticated statistical and machine learning-based methods [15, 18]. Recently, a novel approach has emerged that formulates imputation as a distribution matching problem, first introduced by Muzellec et al. [13]. This approach has attracted considerable attention, with subsequent studies expanding its application to diverse data types and scenarios, including transformed distribution matching for improved accuracy [23], convexified matching for individualized inference [10], and time-series imputation [20].

Distribution matching, also referred to as distribution alignment, is a general technique used to align the distribution of one dataset with that of a reference dataset. The core idea is to minimize the differences between the two distributions, allowing the source data to better reflect the characteristics of the target data. This method is widely applied in various domains, including domain adaptation [5, 14], representation learning [2], and generative modeling [9]. In data imputation, distribution matching aims to recover missing values by making the distribution of incomplete data resemble that of a complete or reference dataset [13]. This alignment ensures the imputed values follow the statistical patterns observed in the data.

While OT-Impute, the distribution matching-based imputation method proposed in [13], achieves high accuracy, it operates entirely automatically, without leveraging human expertise—an element that can significantly enhance the imputation process.¹ In practice, data cleaning tasks, including imputation, often require user verification of cleaned values, especially when data quality is critical or in sensitive applications [4, 11, 22]. Active data cleaning addresses this need by incorporating user feedback to guide automated methods, focusing on problematic areas and selectively querying the most uncertain or error-prone values to maximize user effort's efficiency. Combining active strategies with imputation can improve data quality while efficiently utilizing human input.

In this paper, we introduce active data imputation methods based on distribution matching, extending OT-Impute to incorporate user interaction. A key challenge in this setting is selecting which missing values to present to the user. To address this, we estimate the confidence of imputed values and use it to identify attributes likely to be incorrectly imputed. Specifically, we perform multiple runs of OT-Impute and calculate the standard deviation of imputed values for each attribute value. We then use the exponential decay of the standard deviation as a normalized measure to define the confidence score, where scores close to 0 indicate low confidence and scores close to 1 indicate high confidence. We evaluate these confidence scores using the Brier score, demonstrating that they correlate well with the probability of correct imputation. Notably, the confidence

This work is licensed under the Creative Commons BY-NC-ND 4.0 International License. Visit <https://creativecommons.org/licenses/by-nc-nd/4.0/> to view a copy of this license. For any use beyond those covered by this license, obtain permission by emailing info@vldb.org. Copyright is held by the owner/author(s). Publication rights licensed to the VLDB Endowment.

Proceedings of the VLDB Endowment, ISSN 2150-8097.

¹OT-Impute stands for optimal transport imputer, as it uses distribution distance measures from optimal transport theory. See Section 2 for details.

scores correlate with imputation error under MCAR and MAR across all benchmarks, indicating that low confidence is a reliable signal of imputation error in OT-Impute. However, under MNAR, this correlation does not hold, revealing the inherent challenges of MNAR, where OT-Impute can produce incorrect imputations with high confidence or accurate imputations with low confidence.

Based on our confidence scores, we design Batch-Impute, our first active data imputation method. Batch-Impute runs OT-Impute multiple times, computes the average of the imputed values from the runs as the final imputation, and selects a limited number of low-confidence imputations (based on a fixed budget) to query the user. We evaluate Batch-Impute against several baselines that select values randomly after imputation, while different imputation methods, including OT-Impute, are applied before querying the user. Our results show that Batch-Impute outperforms these baselines, including state-of-the-art imputation methods.

A technical challenge in applying Batch-Impute and computing our confidence scores is the computational cost of running OT-Impute multiple times. To overcome this, we propose an optimized version of Batch-Impute, denoted BatchOP-Impute, which employs dropout noise injection. Originally introduced for neural network regularization [16], dropout randomly masks units during training. We adapt this concept by injecting dropout noise during a single run of OT-Impute, thereby approximating the variability of imputed values without multiple runs. This single stochastic pass serves as an efficient proxy for confidence values, significantly reducing computational costs while maintaining accuracy in detecting low-confidence imputations. We experimentally compare BatchOP-Impute with Batch-Impute in terms of accuracy and runtime, demonstrating its efficiency gains.

Building on Batch-Impute, we develop an iterative active imputation method, Iter-Impute, which distributes the budget across multiple iterations. In each iteration, Batch-Impute is applied to fix selected attribute values, followed by imputing the remaining missing values. Our experiments show that Iter-Impute outperforms Batch-Impute, highlighting the advantages of allocating the budget dynamically and the impact of early versus delayed budget use within Batch-Impute.

Our contributions are as follows:

- We introduce a confidence score for imputed values in OT-Impute, efficiently computed using dropout noise, significantly reducing the computational cost compared to repeated runs of OT-Impute.
- We propose two active imputation methods: Batch-Impute, which selects low-confidence imputations in a single step, and Iter-Impute, an iterative method that dynamically allocates the budget across multiple iterations to progressively improve imputation accuracy.
- We evaluate our methods through comprehensive experiments on real-world datasets, demonstrating that Iter-Impute consistently outperforms both Batch-Impute and random selection baselines in terms of imputation accuracy.

2 BACKGROUND

We now present the background on active imputation and OT-Impute.

2.1 Active Imputation Setting

Active imputation is a strategy for handling missing data that combines automated imputation with selective user interaction to improve accuracy. In this setting, an *oracle*—representing a human expert or a trusted external source—can provide the correct value for any queried missing entry. Since querying the oracle is expensive, we assume a fixed budget β that limits the number of oracle queries.

The objective of active imputation is to maximize the accuracy of the imputed dataset while staying within this fixed oracle budget β .

Formally, let $\mathcal{D}^* \in \mathbb{R}^{n \times d}$ be the complete (unknown) dataset, and let $\mathcal{D} \in \mathbb{R}^{n \times d}$ be the observed version with missing entries. We denote the set of missing cells in $\mathcal{D}$ as $\mathcal{M} \subseteq [n] \times [d]$. An *imputation method* $\mathcal{A}$ produces a completed dataset $\hat{\mathcal{D}} = \mathcal{A}(\mathcal{D}, \mathcal{Q})$, where $\mathcal{Q} \subseteq \mathcal{M}$ is the set of entries queried from the oracle, with $|\mathcal{Q}| \leq \beta$, and each $(i, j) \in \mathcal{Q}$ is replaced by the corresponding true value from $\mathcal{D}^*$. The remaining missing entries are imputed by the method based on observed and queried values.

The goal is to design a strategy for selecting $\mathcal{Q}$ and imputing the rest of $\mathcal{M} \setminus \mathcal{Q}$ such that the imputation error

$$\text{Error}(\hat{\mathcal{D}}, \mathcal{D}^*) = \sum_{(i,j) \in \mathcal{M}} \ell(\hat{\mathcal{D}}_{ij}, \mathcal{D}_{ij}^*)$$

is minimized, where ℓ is an appropriate loss function, such as absolute or squared error for numerical data.

The budget β can be spent all at once (batch mode), or gradually over multiple rounds (iterative mode), where early queries can influence subsequent imputations and query decisions. Designing effective active imputation methods thus involves estimating uncertainty, prioritizing queries, and integrating user input to improve downstream accuracy under limited human effort.

2.2 OT-Based Distribution Matching Imputation

Optimal Transport (OT) provides a principled way to measure the distance between probability distributions [19]. Given two empirical distributions $P = \sum_{i=1}^n p_i \delta_{x_i}$ and $Q = \sum_{j=1}^m q_j \delta_{y_j}$, the OT distance is defined as the solution to:

$$\text{OT}(P, Q) = \min_{\gamma \in \Pi(P, Q)} \sum_{i,j} \gamma_{i,j} c(x_i, y_j)$$

where $c(x_i, y_j)$ is a cost function measuring the distance between x_i and y_j , and $\Pi(P, Q)$ denotes the set of couplings (joint distributions) with marginals P and Q .

To improve computational tractability, Cuturi [6] introduced an entropic regularization term, resulting in the *Sinkhorn divergence*, a smooth and efficiently computable approximation of OT:

$$\text{OT}_\epsilon(P, Q) = \min_{\gamma \in \Pi(P, Q)} \sum_{i,j} \gamma_{i,j} c(x_i, y_j) + \epsilon \sum_{i,j} \gamma_{i,j} \log \gamma_{i,j}$$

$$\text{Sinkhorn}_\epsilon(P, Q) = \text{OT}_\epsilon(P, Q) - \frac{1}{2} \text{OT}_\epsilon(P, P) - \frac{1}{2} \text{OT}_\epsilon(Q, Q)$$

In the OT-Impute method [13], imputation is framed as minimizing the expected Sinkhorn divergence between randomly drawn mini-batches from the incomplete dataset. Let $\theta \in \mathbb{R}^{|\mathcal{M}|}$ denote the learnable variables corresponding to the missing entries. Initially, each θ_{ij} for $(i, j) \in \mathcal{M}$ is initialized with a column-wise mean plus

small noise. At each iteration, two mini-batches B_1 and B_2 are sampled from the dataset (with imputed values), and the following loss is minimized:

$$\min_{\theta} \mathbb{E}_{(B_1, B_2)} [\text{Sinkhorn}_{\epsilon}(B_1, B_2)]$$

The imputation parameters are updated using gradient descent:

$$\theta \leftarrow \theta - \eta \cdot \nabla_{\theta} \mathcal{L}_{\text{Sinkhorn}}(B_1, B_2),$$

where η is the learning rate. RMSProp [17] or other adaptive optimizers are typically used to improve convergence. The final imputed dataset $\hat{\mathcal{D}}$ is constructed by replacing each missing entry $(i, j) \in \mathcal{M}$ in $\mathcal{D}$ with $\theta_{i,j}$, and keeping observed values unchanged.

To further stabilize training and generalization, the optimization employs random batch pairing, early stopping, and regularization. As OT-Impute relies on stochastic batch sampling and non-convex optimization, different runs may produce different imputations for the same missing entries. This variability is later exploited for confidence estimation, as discussed in Section 3.2.

3 CONFIDENCE-AWARE IMPUTATION WITH EFFICIENT APPROXIMATION

In this section, we describe our confidence estimation and active imputation methods.

3.1 Confidence Estimation via Multiple Runs

Given that OT-Impute is a stochastic optimization method—due to random initialization and batch sampling—the imputed values for missing entries can vary across different runs, especially for entries with weak statistical signals. This variability provides a natural basis for estimating *confidence* in the imputed values.

Let $\theta_{i,j}^{(r)}$ denote the imputed value for missing entry $(i, j) \in \mathcal{M}$ obtained from the r -th independent run of OT-Impute. For each $(i, j) \in \mathcal{M}$, we compute the empirical standard deviation:

$$\sigma_{i,j} = \sqrt{\frac{1}{R} \sum_{r=1}^R \left(\theta_{i,j}^{(r)} - \bar{\theta}_{i,j} \right)^2}, \quad \text{where } \bar{\theta}_{i,j} = \frac{1}{R} \sum_{r=1}^R \theta_{i,j}^{(r)}$$

This standard deviation $\sigma_{i,j}$ captures the uncertainty of the imputation at cell (i, j) . A low $\sigma_{i,j}$ indicates stable predictions across runs (high confidence), while a high $\sigma_{i,j}$ suggests inconsistency and low confidence.

To normalize uncertainty into a meaningful score, we apply an exponential decay transformation:

$$c_{i,j} = \exp(-\sigma_{i,j})$$

This maps $\sigma_{i,j}$ to the interval $(0, 1]$, where high-confidence values are close to 1 and low-confidence values are close to 0. The exponential decay emphasizes distinctions among uncertain values, making it easier to identify the most unreliable imputations—ideal candidates for oracle querying under a budget constraint.

While effective, this multi-run approach is computationally expensive. In the next subsection, we introduce a more efficient alternative using stochastic dropout to approximate confidence scores in a single run.

3.2 Dropout for Efficient Confidence Estimation

To reduce the cost of repeated runs for confidence estimation, we propose a stochastic approximation method based on *dropout noise*, inspired by Monte Carlo Dropout [8]. Rather than recomputing standard deviations over multiple independent runs of OT-Impute, we inject noise into the optimization process to simulate variability within a single run.

Let $\theta \in \mathbb{R}^{|\mathcal{M}|}$ denote the vector of learnable parameters corresponding to the missing entries. During optimization, we apply multiplicative dropout noise $\mathbf{z} \sim \text{Bernoulli}(1-\delta)^{|\mathcal{M}|}$, masking each coordinate of θ independently. The resulting perturbed imputation vector is $\tilde{\theta} = \theta \odot \mathbf{z}$, where $\odot$ denotes elementwise multiplication.

The Sinkhorn loss is then evaluated using the imputed dataset formed by inserting $\tilde{\theta}_{i,j}$ at each $(i, j) \in \mathcal{M}$. The update rule becomes:

$$\theta \leftarrow \theta - \eta \cdot \nabla_{\theta} \mathcal{L}_{\text{Sinkhorn}}(\tilde{\theta}),$$

where η is the learning rate. This dropout-perturbed update implicitly induces variability across imputed values, approximating the distribution that would be obtained by running OT-Impute multiple times.

To estimate confidence, we collect imputed values $\tilde{\theta}_{i,j}$ over multiple forward passes or optimization epochs and compute their empirical variance for each $(i, j) \in \mathcal{M}$. This variance serves as a proxy for uncertainty. As shown in our experiments, this method significantly reduces computational cost—from $O(R \cdot T)$ for R runs with T steps each to $O(T)$ for a single stochastic optimization—while still yielding informative confidence scores.

3.3 Iterative Imputation with User Interaction

Active imputation incorporates user input to iteratively improve imputation accuracy. In our setting, we assume access to an oracle that can reveal the true value of any missing entry $(i, j) \in \mathcal{M}$. The method proceeds in K iterations, allocating a fixed query budget β over time.

Let $\theta^{(k)}$ denote the current imputed values at iteration k , and let $c_{i,j}^{(k)}$ be the confidence score for each $(i, j) \in \mathcal{M}$, estimated via the dropout approximation in Section 3.2. At each iteration, we select a batch $Q^{(k)} \subseteq \mathcal{M}$ of the lowest-confidence entries such that:

$$\sum_{k=1}^K |Q^{(k)}| \leq \beta$$

The values in $Q^{(k)}$ are queried from the oracle and incorporated into the dataset as fixed (non-learnable) values. The imputer is then retrained on the updated dataset, and a new imputation vector $\theta^{(k+1)}$ is produced, along with updated confidence scores.

This process repeats until the budget β is exhausted or a stopping criterion (e.g., confidence saturation or convergence of imputation error) is met. By dynamically integrating newly revealed values, the method adapts to the uncertainty landscape and prioritizes queries that most improve future imputations.

Our experiments demonstrate that this iterative approach significantly outperforms one-shot querying strategies, achieving higher accuracy with more efficient use of human feedback.

4 EXPERIMENTS

Our experiments pursue four main objectives: (1) to assess the effectiveness of confidence scores in identifying imputation errors (Section 4.2); (2) to evaluate the performance of Batch-Impute compared to random selection of query values (Section 4.3); (3) to examine the impact of dropout-based optimization in BatchOP-Impute (Section 4.4); and (4) to demonstrate the effectiveness of iterative querying in Iter-Impute and analyze how the number of iterations affects imputation accuracy (Section 4.5).

4.1 Experimental Setup

We summarize the experimental setup and assumptions. Code, data, and additional results are available in our GitHub repository [1].

Datasets: We use several popular real-world tabular datasets, as summarized in Table 1. The table also includes imputation complexity, which reflects the relative difficulty of imputing missing values based on data type heterogeneity. Cat. stands for Categorical and Cont. for Continuous attributes.

- **adult:** UCI Census Income [7], containing demographic and employment-related features for income prediction.
- **cancer:** Breast Cancer Wisconsin (Diagnostic) [21], containing numerical features derived from breast mass samples.
- **credit:** German Credit [7], with a mix of categorical and numerical features related to credit risk assessment.
- **wine:** Wine [7], containing continuous attributes from the chemical analysis of Italian wines.
- **disease:** Disease [7], a medical dataset for predicting disease presence based on health-related features.

Dataset	#Rec.	#Attr.	Type	Complexity
Adult	48,842	14	Cat./Cont.	Moderate
Cancer	569	30	Cont.	Low
Credit	1,000	20	Cat./Num.	High
Wine	178	13	Cont.	Low
Disease	1,416	35	Mixed	Moderate

Table 1: Statistics of the datasets.

For all datasets, we preprocess them by 1) encoding categorical variables using integer codes, 2) scaling all feature values to the range $[0, 1]$ using min-max normalization, and 3) inject missing values synthetically using three missingness mechanisms [12] of **MCAR** (missing completely at random), **MAR** (missing at random), and **MNAR** (missing not at random). The result of step 2 serves as our ground truth and we use the result of step 3 for the imputation methods. More details of our missing value injection are in our repository [1].

Parameters and Measures: For each setting, we control the **missingness rate** ϵ and inject missing attribute values according to the mechanism type. For experiments in Sections 4.3 and 4.5, we simulate human supervision using an oracle that can reveal the true value of a missing attribute value. For a fixed **budget** β , the imputation method can query $\beta \times$ the number of missing values. For our optimization, we look into **dropout noise** δ . We use **MAE**

(mean absolute error) between the imputed and ground-truth values, computed only on missing entries to evaluate the outcome of the imputation methods. We use the **Brier score** to assess how well the estimated confidence values reflect the correctness of imputed values. It is a proper scoring rule for probabilistic predictions and measures confidence calibration. Given an imputed value x_i with confidence $c_i \in [0, 1]$ and a binary correctness indicator $y_i \in \{0, 1\}$ (where $y_i = 1$ if correct), the Brier score [3] is defined as $B = \frac{1}{n} \sum_{i=1}^n (c_i - y_i)^2$. Lower scores indicate better calibration, with 0 being perfect. A score around 0.25 corresponds to random guessing with $c_i = 0.5$, offering a baseline for comparison.

Imputation Methods: In addition to the methods already described (OT-Impute, Batch-Impute, BatchOP-Impute, and Iter-Impute), we use the following imputation methods in our experiments:

- **KNN:** k-Nearest Neighbors imputation, uses the k nearest records to impute missing values in a record.
- **Mean:** Mean imputation [12], which replaces missing values with the mean of the observed values within a variable.
- **MICE:** Multiple Imputation by Chained Equations (MICE) [18]. An iterative approach that generates multiple imputed datasets.

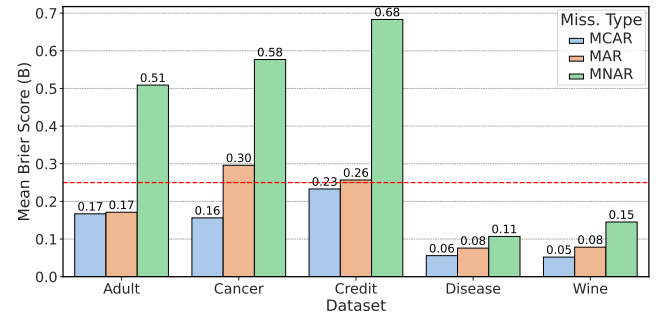
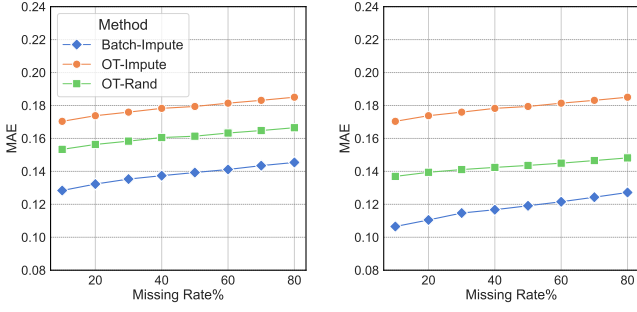


Figure 1: The Brier score, B , for different datasets and missingness types. The red dashed line at $B = 0.25$ represents the random guessing baseline.

4.2 Confidence Score and Imputation Error

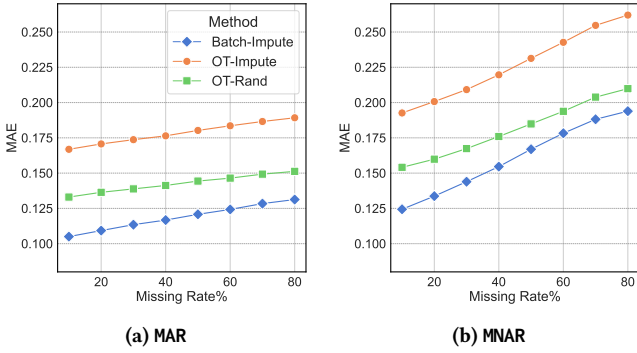
Figure 1 reports the Brier scores (B) for different datasets and missingness mechanisms. A Brier score near 0.25 corresponds to random guessing with confidence $c_i = 0.5$, serving as a baseline. Scores below 0.25 indicate that the confidence estimates provide useful signal about imputation correctness.

Under MCAR, most datasets achieve Brier scores well below 0.25, showing that confidence scores reliably correlate with imputation error. MAR also shows strong correlation, except for cancer and credit. In contrast, under MNAR, results are mixed: while wine and disease maintain relatively low Brier scores, datasets like adult, credit, and cancer show high scores, indicating unreliable confidence estimates. These differences reflect dataset characteristics. wine and disease are low-dimensional, mostly numerical, and exhibit homogeneous patterns—conditions under which OT-Impute produces stable imputations and well-calibrated confidence scores. In contrast, adult and credit are more heterogeneous and high-dimensional, with mixed attribute types. This leads OT-Impute



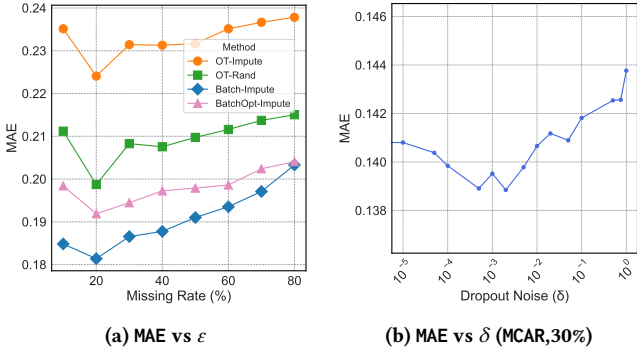
(a) $\beta = 0.1$ (b) $\beta = 0.2$

Figure 2: MAE vs ϵ (missing rate) in adult (MCAR) .



(a) MAR (b) MNAR

Figure 3: MAE vs ϵ (missing rate) in adult ($\beta = 0.2$) .



(a) MAE vs ϵ (b) MAE vs δ (MCAR,30%)

Figure 4: Batch-Impute and BatchOP-Impute in credit (MCAR)

to produce confident but incorrect imputations or uncertain but correct ones, especially under MNAR. The experiment shows confidence scores are more informative when the data is homogeneous, low-dimensional, and features are strongly correlated.

4.3 Evaluation of Batch-Impute

Figure 2 compares the performance of Batch-Impute, OT-Impute, and OT-Impute followed by random oracle queries (OT-Rand) across different budgets (β) and missing rates (ϵ). Both OT-Rand and Batch-Impute outperform OT-Impute by leveraging oracle feedback, achieving

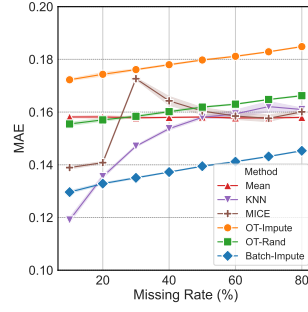


Figure 5: adult (MCAR)

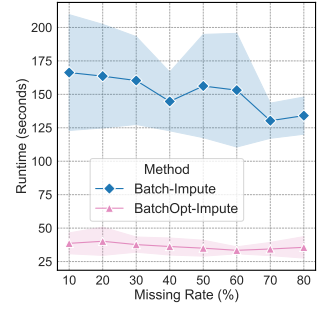


Figure 6: credit (MCAR)

lower MAE. Batch-Impute consistently outperforms OT-Rand, confirming that its confidence scores are effective for identifying uncertain imputations. By prioritizing low-confidence imputed attribute values, Batch-Impute achieves more corrections of imputation error than random selection in OT-Rand.

As the missing rate increases, imputation error rises for all methods due to reduced information available to OT-Impute. Figures 2a and 2b show that increasing the budget improves Batch-Impute’s performance by enabling more informed queries. However, at high missingness levels, limited observed data restricts accuracy improvements, and the gap between Batch-Impute and OT-Rand narrows—though Batch-Impute still maintains a consistent advantage.

Figures 3a and 3b show similar trends under MAR and MNAR. Under MNAR, the performance of Batch-Impute declines more noticeably and approaches that of OT-Rand at high missingness, reflecting the reduced reliability of confidence scores in this setting (see Section 4.2).

Figure 5 compares Batch-Impute with standard imputation methods under MCAR, where each method is followed by random oracle queries. Batch-Impute consistently achieves the lowest MAE across all missingness levels. OT-Impute shows the highest error, as it does not incorporate user feedback. However, its performance improves notably when combined with active querying. KNN performs well at low missingness but degrades quickly as missingness increases, eventually performing worse than OT-Rand.

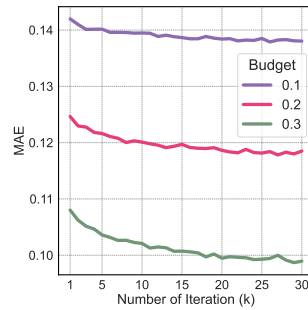


Figure 7: cancer (MCAR,30%).

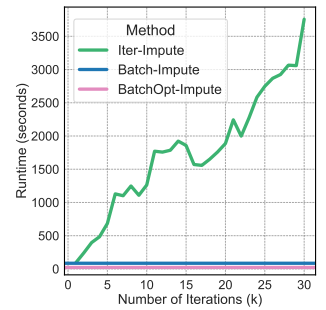


Figure 8: cancer (MCAR,30%).

4.4 Optimization with Dropout Noise

Figure 6 compares runtime versus missing rate for Batch-Impute and BatchOP-Impute. BatchOP-Impute is about four times faster, as it avoids repeated optimization and performs only one pass, whereas Batch-Impute uses four. Figure 4a shows that although BatchOP-Impute is less accurate at low missingness, its performance converges to that of Batch-Impute as missingness increases, making it a practical trade-off between accuracy and efficiency on sparse datasets. Figure 4b highlights the effect of the dropout rate. Moderate noise levels yield the best results, while very low or high dropout rates degrade performance. This shows the need for tuning to get reliable confidence estimation.

4.5 Evaluation of Iter-Impute

Figure 7 shows that performance consistently improves with each iteration in Iter-Impute, as more ground truth values are incorporated. Higher budgets further accelerate this improvement, leading to faster and more substantial gains in imputation accuracy. Figure 8 compares the runtime of Iter-Impute, Batch-Impute, and BatchOP-Impute. The runtime of Iter-Impute increases sharply with the number of iterations due to repeated retraining. In contrast, Batch-Impute and BatchOP-Impute maintain constant runtimes regardless of the number of iterations, since they operate in a single pass without repeated querying.

5 CONCLUSION AND FUTURE WORK

This paper introduced a confidence-based framework for active data imputation using distribution matching. We proposed methods to estimate the confidence of imputed values, including a dropout-based approximation for efficient computation. Using these scores, we developed Batch-Impute and Iter-Impute, two active querying strategies for guiding limited user interaction. Experiments show that confidence scores effectively identify unreliable imputations under MCAR and MAR, enabling more accurate and efficient querying than random selection. Batch-Impute improves accuracy by focusing on low-confidence entries, while BatchOP-Impute offers a faster alternative with similar performance at high missingness. Iter-Impute further improves accuracy by incorporating user feedback across iterations, albeit with higher runtime. These results demonstrate that confidence-guided active imputation is a practical and effective approach for semi-automated data cleaning with limited human input.

Future research directions fall into two categories. The first involves improving OT-Impute and the specific confidence-based methods in this paper. While our dropout-based approximation reduces runtime, its accuracy depends on noise settings. More principled uncertainty estimation techniques—such as Bayesian optimization or ensembles—could yield better results. Extending OT-Impute to handle categorical or mixed-type data more robustly, beyond simple encodings, would increase applicability. Additionally, future methods could support adaptive budgeting or cost-aware querying, instead of assuming a fixed, uniform-cost budget.

The second direction is broader and concerns confidence estimation in data cleaning more generally. In OT-Impute, computing confidence was relatively straightforward since imputed values are

continuous and optimized directly; the main challenge was computational. In contrast, applying confidence estimation to other cleaning or imputation methods—such as rule-based, non-parametric, or probabilistic models—may require new formulations, as these methods often lack differentiability or probabilistic outputs.

Beyond active querying, confidence scores have broader applications. They can help prioritize manual review, explain downstream decisions based on cleaned data, or guide selective retraining by flagging low-confidence records. More generally, confidence estimation can enhance semi-automated cleaning workflows by making uncertainty explicit and actionable. Extending this idea to other tasks—such as error detection, entity resolution, or deduplication—can support more transparent and robust data preparation pipelines.

REFERENCES

- [1] 2025. Active Imputer. https://github.com/ZarinTahia/active_imputer.
- [2] Martin Arjovsky, Soumith Chintala, and Léon Bottou. 2017. Wasserstein GAN. In *ICML*.
- [3] Glenn W. Brier. 1950. Verification of Forecasts Expressed in Terms of Probability. *Monthly Weather Review* 78, 1 (1950), 1–3.
- [4] Xu Chu, Ihab F Ilyas, Sanjay Krishnan, and Jiannan Wang. 2016. Data Cleaning: Overview and Emerging Challenges. In *SIGMOD*.
- [5] Nicolas Courty, Rémi Flamary, Devis Tuia, and Alain Rakotomamonjy. 2017. Joint distribution optimal transportation for domain adaptation. In *NeurIPS*.
- [6] Marco Cuturi. 2013. Sinkhorn distances: Lightspeed computation of optimal transport. In *NeurIPS*.
- [7] Dheeru Dua and Casey Graff. 2019. UCI Machine Learning Repository. <http://archive.ics.uci.edu/ml> University of California, Irvine, School of Information and Computer Sciences.
- [8] Yariv Gal and Zoubin Ghahramani. 2016. Dropout as a Bayesian approximation: Representing model uncertainty in deep learning. In *ICML*.
- [9] Aude Genevay, Gabriel Peyré, and Marco Cuturi. 2018. Learning generative models with sinkhorn divergences. In *AISTATS*.
- [10] Y. Hur and T. Liang. 2024. A Convexified Matching Approach to Imputation and Individualized Inference. *arXiv preprint arXiv:2407.05372* (2024). <https://arxiv.org/abs/2407.05372>
- [11] Sanjay Krishnan, Jiannan Wang, Eugene Wu, Michael J Franklin, and Kenneth Goldberg. 2016. ActiveClean: Interactive Data Cleaning While Learning. In *ICDE*.
- [12] Roderick J.A. Little and Donald B. Rubin. 2002. *Statistical Analysis with Missing Data* (2nd ed.). John Wiley & Sons.
- [13] Boris Muzellec, Julie Josse, Claire Boyer, and Marco Cuturi. 2020. Missing data imputation using optimal transport. In *ICML*. 7130–7140.
- [14] Ievgen Redko, Nicolas Courty, Rémi Flamary, and Devis Tuia. 2017. Theoretical analysis of domain adaptation with optimal transport. *ECML PKDD* (2017).
- [15] Donald B Rubin. 1976. Inference and missing data. *Biometrika* 63, 3 (1976), 581–592.
- [16] Nitish Srivastava, Geoffrey Hinton, Alex Krizhevsky, Ilya Sutskever, and Ruslan Salakhutdinov. 2014. Dropout: A simple way to prevent neural networks from overfitting. *JMLR* 15, 1 (2014), 1929–1958.
- [17] Tijmen Tieleman and Geoffrey Hinton. 2012. Lecture 6.5 - RMSProp: Divide the gradient by a running average of its recent magnitude. Course: Neural Networks for Machine Learning. Lecture 6.5, University of Toronto.
- [18] Stef Van Buuren and Karin Groothuis-Oudshoorn. 2011. MICE: Multivariate Imputation by Chained Equations in R. *Journal of Statistical Software* 45, 3 (2011), 1–67. <https://doi.org/10.18637/jss.v045.i03>
- [19] Cédric Villani. 2008. *Optimal Transport: Old and New*. Springer.
- [20] Hao Wang, Haoxuan Li, Xu Chen, Mingming Gong, Zhichao Chen, et al. [n.d.]. Optimal Transport for Time Series Imputation. In *ICLR*.
- [21] William H. Wolberg and Olvi L. Mangasarian. 1995. Breast Cancer Wisconsin (Diagnostic) Data Set. *UCI Machine Learning Repository* (1995). [https://archive.ics.uci.edu/ml/datasets/Breast+Cancer+Wisconsin+\(Diagnostic\)](https://archive.ics.uci.edu/ml/datasets/Breast+Cancer+Wisconsin+(Diagnostic))
- [22] Jingbo Wu, Jiannan Li, Sanjay Krishnan, et al. 2023. Data Cleaning: A Survey. *Vldb* (2023).
- [23] He Zhao, Ke Sun, Amir Dezfouli, and Edwin V Bonilla. 2023. Transformed distribution matching for missing value imputation. In *International Conference on Machine Learning*. PMLR, 42159–42186.